Self-interpretation in PTS-rk: a rank obstruction and an indexed approach

October 20, 2026

Abstract

Fixed-rank elementary normalization, together with non-uniform elementary representability, gives a constructive reason to seek rank-indexed self-interpretation of the original annotated PTS-rk. We distinguish the checked normalization theorem from the manuscript-level representability argument, and numeric self-reduction from typed unquotation. Rocq now connects full-PTS normalization to faithful HOAS syntax, and supports a normalization-assisted weak interpretation carrying both syntax and a normal semantic value. Its decoder returns the source normal form, not the unreduced source. Separate rank, single-family and literal-generation obstructions constrain stronger interfaces without excluding sufficiently indexed constructions. A finite closed source-type-indexed PHOAS representation with strong unquotation is now checked for the independent simply typed fragment. Genuinely dependent indexed handlers are also constructed; a separate theorem excludes conflating faithful syntax with semantic type indices. For the full dependent calculus, executable canonicalization now extends to open contexts, with bidirectional original typing transport, hereditary substitution and separate conversion-canonical HOAS type codes. A full dependent interpreter evaluating HOAS or PHOAS syntax internally, rather than projecting a precomputed value, remains open.

1 Why the complexity bounds suggest an indexed interpreter

There is positive computational evidence for a non-uniform self-interpreter. The checked normalization development covers the **full original annotated PTS**, not just a simply typed sublanguage. For a well-typed d-bounded term t in a well-formed context, d bottom-up greedy passes produce its normal form at the same original type. Writing N for its node count and $\mathrm{tower}_0(x) = x$, $\mathrm{tower}_{h+1}(x) = 2^{\mathrm{tower}_h(x)}$, the proved operation-cost bound is

$$\mathrm{cost}(\mathsf{normalize}_d(t)) \leq \mathrm{tower}_{2d+1}(N + 5d + 2).$$

Thus normalization is elementary for each fixed external d. Moreover, every **closed, originally well-typed** term has the computable sufficient bound

$$d(t) = 1 + \mathsf{maxRankLiteral}(t) + \mathsf{nodes}(t).$$

For the documented encoding size s, this bound is at most $2s + 1$ and its traversal costs at most $7s$. These are the public Rocq results `fixed_rank_normalization` and `rank_bound_in_linear_time`. The uniform automatic normalizer has an input-dependent tower-height bound; this does not provide a uniform fixed-height elementary bound on all inputs. The numerical rank bound here is a bound on redex heads, not merely the rank of the final result type. The open-context extension below computes a sufficient bound from both the term and its context declarations; it requires no normalization-success oracle.

The manuscript's computational-model section, theorem label `non-uniform-rep`, supplies the other half of the proposed argument: for each elementary function f there is a rank cost c_f and, at each base rank j, a numeral realizer

$$F_j : \mathcal{N}_{j+c_f} \to \cdots \to \mathcal{N}_{j+c_f} \to \mathcal{N}_j, \qquad \mathcal{N}_j = \Pi X : \mathcal{U}_j.(X \to X) \to X \to X,$$

computing f on canonical Church numerals. The rank cost is computed from an explicit elementary program, using numeral-preserving rank lowerings and composition. It is not a rank inferred from arbitrary extensional black-box knowledge of f. The cited elementary substitution-basis theorem[1] gives the relevant arithmetic basis. A constructive compilation of the fixed-d normalization algorithm would therefore yield an effective external rank budget depending on d, its elementary program and the chosen serialization; no numerical formula for that compiler budget is currently kernel-checked here.

Proposition 1.1 (The complexity-to-self-reduction argument)**.** *Assume the manuscript's non-uniform elementary representability theorem and elementary encoding/decoding of annotated syntax as numbers. For each fixed d and base rank j, there is an original PTS numeral program $S_{d,j} : \mathcal{N}_{j+c_d} \to \mathcal{N}_j$ which, on the numerical code of any originally typed d-bounded term, returns the numerical code of its full beta-normal form.*

Proof. On arbitrary numerical input, parse a raw annotated syntax tree, returning a fixed default for malformed input. Perform d greedy passes and encode the result. For fixed d, the structural operations, serialization and fixed-height bounds give an elementary total function; no type-checker or unbounded search is needed in this algorithm. Apply non-uniform representability to that function. On the independently specified typed, d-bounded domain, the checked normalization theorem supplies correctness. The program is fixed for d, j, not chosen anew for each input term. □

This is a genuine route to an **indexed numeric self-reducer**. Its non-uniformity avoids the fixed-head rank obstruction. However, a number coding a normal form is not an inhabitant of the represented dependent type. A HOAS/PHOAS interpreter still needs actual typed adapters and reification, including context and conversion coherence. Further, normalization yields $\mathsf{nf}(t) =_\beta t$, not reduction back to the original unreduced t. These distinctions determine what the complexity argument proves and what remains to construct.

Verification boundary. Normalization and its stated operation costs are checked. The generic elementary-function compiler is not formalized; `latex/ROCQ_REFERENCES.md` explicitly records this boundary. The displayed modulus construction in the manuscript also needs the local callback correction $r\, f\, x$ in place of $r\, X\, f\, x$, because that record field is already specialized to X. We do not silently treat the uncorrected display as a checked inhabitant. The numerical proposition above is a mathematical conditional, not a new Rocq axiom or a completed internal HOAS interpreter.

2 Question and source

The reference is Matt Brown and Jens Palsberg, *Breaking through the Normalization Barrier: A Self-Interpreter for F-omega*, POPL 2016, pp. 5–17, doi:10.1145/2837614.2837623[2]. The

[1] `https://arxiv.org/abs/2505.23787`

[2] `https://doi.org/10.1145/2837614.2837623`

supplied PDF has filename `2914770.2837623.pdf`; the theorem labels below refer to its text, not to PDF page indices.

Their target is an **unquoter**: an external quotation operation assigns a typed, beta-normal representation $\ulcorner t \urcorner$ to each term t, and an object-language program recovers t from this representation. The strong version requires reduction to the original term, $\mathsf{unquote}\ulcorner t \urcorner \longrightarrow_\beta^* t$, rather than merely conversion to it. Quotation must preserve syntax injectively, with the paper's precise treatment of equivalent type annotations. Returning the quotation of a normal form is a different operation, called a self-reducer.

Section 3.3, Theorems 3.4–3.5, gives the shallow construction. A neutral variable of type $\forall A : *.A \to A$ blocks each application; substituting the polymorphic identity unblocks it. Sections 4–7 replace this with a deeper representation using typed higher-order abstract syntax, extensional instantiation functions, and interpretations of types. Theorems 6.1–6.3 establish typing, normality, and injectivity up to equivalence of type annotations; Theorem 7.4 establishes strong recovery. Section 10 leaves dependent self-interpretation and self-representation without impredicativity as open directions.

These are not impossibility results from strong normalization alone. The paper specifically explains why a numerical diagonalization argument does not automatically apply to typed quotation. The obstruction below instead uses the exact PTS-rk product rule.

3 The original calculus and the rank invariant

All claims use the existing `PTSrk.Syntax.Term nat n`, including lambda annotations, `Typing.typ`, `Ranks.RankSorts`, and original beta reduction. There is no cumulativity, added primitive, eta conversion, or replacement typing judgment. Its sort and product rules are

$$\Gamma \vdash \mathcal{U}_i : \mathcal{U}_0, \qquad \frac{\Gamma \vdash A : \mathcal{U}_a \quad \Gamma, x : A \vdash B : \mathcal{U}_b}{\Gamma \vdash \Pi x : A.B : \mathcal{U}_{\max(a+1,b)}}.$$

The **type rank** of a typed term t is r when $\Gamma \vdash t : T$ and $\Gamma \vdash T : \mathcal{U}_r$. This is the rank of its type, not its largest sort literal or a bound on all redexes inside t. In well-formed contexts it exists, is unique, and is invariant under beta conversion of typed terms. These facts are already proved in `NormalizationComplexity/RedexRanks.v`.

Lemma 3.1 (Application rank)**.** *If* Γ *is well formed and* $f\ u$ *is typed, then* $\mathrm{rk}_\Gamma(f\ u) \le \mathrm{rk}_\Gamma(f)$.

Proof. Invert the typing of the application to obtain $f : \Pi x : A.B$ and $u : A$. If $A : \mathcal{U}_a$ and $B : \mathcal{U}_b$ under the binder, the type of f has rank $\max(a+1, b)$. Substitution preserves $B : \mathcal{U}_b$, so $B[u/x]$ has rank b. Uniqueness of type rank removes any dependence on the chosen derivation or intervening conversion rules. □

Consequently, for any well-typed finite application spine,

$$\mathrm{rk}(E\ a_1 \cdots a_m) \le \mathrm{rk}(E).$$

Arguments may be types, representations, or other terms. Their number and their construction may depend arbitrarily on the represented program. This bound already holds without a computable quotation operation.

Theorem 3.2 (No universal fixed-head evaluator)**.** *There is no closed typed PTS-rk term E such that every closed typed term t is beta-convertible to a well-typed finite application spine headed by E.*

Proof. Define closed types and inhabitants by

$$A_0 = \mathcal{U}_0, \quad A_{r+1} = \Pi x : A_r.\mathcal{U}_0, \qquad v_0 = \mathcal{U}_0, \quad v_{r+1} = \lambda x : A_r.\mathcal{U}_0.$$

The original rules derive $A_r : \mathcal{U}_r$ and $v_r : A_r$ for every r. Suppose E has type rank R. An application spine of E has rank at most R, so invariance under conversion prevents it from being convertible to v_{R+1}. This contradicts universality. □

The Rocq declaration is `RankObstruction.no_universal_fixed_head_evaluator`. Its specification allows arbitrary externally chosen arguments and arbitrary intermediate result types. It assumes neither an encoding nor normality nor injectivity, making the exclusion stronger than failure of one particular encoding.

This excludes a direct port of the actual fixed unquote programs in Sections 3.3 and 7, including implicit type applications. A fixed wrapper chosen once is another fixed E and is excluded as well. Section 3.2 also permits implicit type *abstractions*. If interpreted as arbitrarily many term-dependent outer wrappers, these lie outside the application-spine theorem: external abstraction can raise rank. We do not claim to have excluded every liberal interpretation of that allowance, externally generated interpreter families, or a weaker observational semantics.

4 What the sort cycle does and does not provide

Every $\mathcal{U}_r$ has type $\mathcal{U}_0$, but this does not place every type in $\mathcal{U}_0$. For example $A \to A$ has rank $r+1$ when $A : \mathcal{U}_r$. Thus there is no single sort playing the role of the arrow-closed kind $*$ of System F_ω. Replacing every occurrence of $*$ by $\mathcal{U}_0$ in the paper's code does not produce a typing derivation.

There is, however, an identity at every external rank:

$$\mathsf{I}_r = \Pi A : \mathcal{U}_r.\, A \to A, \qquad \mathsf{id}_r = \lambda A : \mathcal{U}_r.\lambda x : A.x, \qquad \mathsf{I}_r : \mathcal{U}_{r+1}, \quad \mathsf{id}_r : \mathsf{I}_r.$$

The numerical index r is part of generated syntax. Rocq quantification over $r : \mathbb{N}$ produces a family of original PTS terms; it is not an internal PTS quantifier over ranks. In particular an object numeral cannot simply be substituted for the sort label in $\mathcal{U}_r$.

For a blocker rank r and an output-type rank s, define

$$\begin{aligned} \mathsf{Code}_r(A) &= \mathsf{I}_r \to A, \\ \mathsf{D}_{r,s} &= \lambda A : \mathcal{U}_s.\lambda q : \mathsf{Code}_r(A).\, q\, \mathsf{id}_r, \\ T_{r,s} &= \Pi A : \mathcal{U}_s.\, \mathsf{Code}_r(A) \to A. \end{aligned}$$

The exact original judgments, proved in `RankedDecoder.v`, are

$$\begin{aligned} A : \mathcal{U}_s \implies \mathsf{Code}_r(A) &: \mathcal{U}_{\max(r+2,s)}, \\ \mathsf{D}_{r,s} : T_{r,s}, \qquad T_{r,s} &: \mathcal{U}_{\max(r+3,s+1)}. \end{aligned}$$

For any typed code $q : \mathsf{Code}_r(A)$, $\mathsf{D}_{r,s}\, A\, q \longrightarrow_\beta^* q\, \mathsf{id}_r$. The raw packet $p_r(t) = \lambda i : \mathsf{I}_r.t$ (with the necessary de Bruijn shift) satisfies $\mathsf{D}_{r,s}\, A\, p_r(t) \longrightarrow_\beta^* t$ in three beta steps. This is a typed retraction for every original typed term, including dependent terms. It is **not a quotation theorem**: $p_r(t)$ retains every source redex. The declaration `typed_ranked_retraction` deliberately makes only the retraction claim.

The theorem `decoder_type_not_in_result_universe` formalizes a failure of self-specialization. The decoder cannot be instantiated at its own type in its output universe: $T_{r,s}$ has rank at least $s + 1$, hence cannot also inhabit $\mathcal{U}_s$. Increasing an external index is therefore a real change of program, not the paper's self-application of one polymorphic interpreter.

5 HOAS replacement for the complete annotated syntax

Terms and types share the original syntax. The former first-order Church AST is now replaced by a higher-order encoding: source variables become target variables, and source binding becomes actual target lambda binding. Original product domains and lambda annotations remain explicit data. This represents every raw annotated source tree, not only well-typed ones.

Fix a carrier rank r and an external source scope size n. Write $X^n \to X$ for n curried arguments of type X, with $X^0 \to X = X$, and put

$$\begin{aligned} H(X) &= X \to (X \to X) \to X, \\ \mathsf{HOAS}_{r,n} &= \Pi X : \mathcal{U}_r.\, X \to (X \to X) \to (X \to X) \\ &\qquad \to H(X) \to H(X) \to (X \to X \to X) \to (X^n \to X). \end{aligned}$$

The six handlers are numeral zero z, numeral successor s, sort u, product p, lambda l, and application a. There is no variable constructor. For an environment ρ assigning carrier terms to source variables, the executable fold is

$$\begin{aligned} F_\rho(\mathcal{U}_k) &= u\,(s^k z), & F_\rho(x) &= \rho(x), \\ F_\rho(\Pi x : A.B) &= p\,(F_\rho(A))\,(\lambda x : X.F_{\rho,x}(B)), \\ F_\rho(\lambda x : A.b) &= l\,(F_\rho(A))\,(\lambda x : X.F_{\rho,x}(b)), \\ F_\rho(f\,t) &= a\,(F_\rho(f))\,(F_\rho(t)). \end{aligned}$$

The binder clauses shift every handler and outer environment entry and map the newly bound source variable to the new target variable. This is HOAS inside the original PTS syntax, not a Rocq function hidden in an abstract syntax constructor. The closed quote $C_{r,n}(t)$ abstracts over X, z, s, u, p, l, a, then over the n carrier variables needed to close its source environment. For example, at source scope zero,

$$\begin{aligned} C_{r,0}(\lambda x : \mathcal{U}_k.x) = \lambda X : \mathcal{U}_r.\lambda z : X.\lambda s : X \to X. \\ \lambda u : X \to X.\lambda p : H(X).\lambda l : H(X). \\ \lambda a : X \to X \to X.\ l\,(u\,(s^k z))\,(\lambda x : X.x). \end{aligned}$$

Proposition 5.1 (Normal, faithfully encoded raw HOAS)**.** *For every original $t : \mathsf{Term}\ \mathbb{N}\ n$ and every external r, the original rules derive $\vdash C_{r,n}(t) : \mathsf{HOAS}_{r,n}$ and $\vdash \mathsf{HOAS}_{r,n} : \mathcal{U}_{r+3}$. The quotation is fully beta-normal. At each fixed source scope n,*

$$C_{r,n}(t) =_\beta C_{s,n}(t') \quad \Longleftrightarrow \quad r = s \text{ and } t = t'$$

as literal annotated source syntax.

The binder handler $H(X)$ has rank $r + 2$, so accepting this handler raises the code type to rank $r + 3$. This is one higher than the replaced first-order representation. Additional carrier-variable arguments do not raise that bound. The existence endpoint in `ChurchSyntax` is `every_original_term_has_church_code`. The fidelity theorem is `ChurchSyntaxFaithful.quote_conversion_iff`. Normality includes the generated lambda annotations. Injectivity is proved by structural inversion under an injective variable environment, not inferred from evaluation correctness. It therefore retains distinctions between a reducible source annotation and its contractum.

Applying a quotation to its carrier and six handlers performs seven administrative beta steps:

$$C_{r,n}(t)\, X\, z\, s\, u\, p\, l\, a \longrightarrow_\beta^* \lambda x_1 : X. \cdots \lambda x_n : X. F_{\vec{x}}(t).$$

`ChurchSyntaxFold.run_fold` establishes this reduction, and `run_typing` derives the original typing of the actual application with result type $X^n \to X$. No subject expansion is used. With $n = 0$ the result is the unwrapped fold. Source sort labels remain numeral data; no lower bound on r is imposed by their magnitude.

Terminological boundary. This is a PTS-typed encoding of raw syntax. Its carrier does not depend on the source term's type. In the terminology of the paper it is an *untyped source representation*, despite its actual target typing theorem. It is not the source-type-indexed construction discussed next. No claim is made that every inhabitant of its code type arises from source syntax, or that its arbitrary algebras perform typed unquotation. Instantiating its carrier by another code type requires $r = s + 3$, without cumulativity or an internal rank quantifier.

6 Which indices a raw-HOAS decoder must retain

There are three different quantities: a redex bound d, the exact rank s of the result type, and the largest literal sort label ℓ in the source. The conservative bound $d(t)$ in the introduction includes literal labels. An arbitrary sufficient redex bound need not do so: every $\mathcal{U}_k$ is already normal, has redex bound zero, and has type $\mathcal{U}_0$, of rank zero, regardless of k.

The raw HOAS quote deliberately turns these labels into numeral data. In fact, `LiteralObstruction.quote_literal_exact` proves

$$\mathsf{maxRankLiteral}(C_{r,n}(t)) = r.$$

Theorem 6.1 (Literal-generation obstruction)**.** *For any fixed closed raw term E and carrier index r, if $E\, C_{r,0}(\mathcal{U}_k) =_\beta \mathcal{U}_k$, then*

$$k \leq \max\{\mathsf{maxRankLiteral}(E), r\}.$$

Consequently no such E unquotes all normal sorts from these codes, even when the source redex bound and result-type rank are both zero.

Proof. Original beta reduction never creates a larger literal sort label. Conversion to the normal sort $\mathcal{U}_k$ implies reduction to it by the common-reduct theorem. The applied code contains only the literal labels of E and the single carrier label r. Taking k one greater than their maximum gives a counterexample. No typing premise on E is needed. □

The theorem `no_fixed_type_argument_unquoter` also permits the ordinary type argument $\mathcal{U}_0$ before the raw code. This argument supplies none of the missing k. Thus indexing an interpreter by **only** redex bound and result-type rank does not solve raw-code unquotation. A family whose generated syntax additionally depends on a literal ceiling ℓ, or on the conservative bound which includes it, is not excluded. Neither is a representation carrying additional literal-bearing semantic data. The obstruction does not concern numeric code-to-code self-reduction: an output code can represent k by data without generating the literal $\mathcal{U}_k$.

7 Full-PTS normalization connected to HOAS

The executable metalevel construction in `RankedNormalization` is

$$\mathsf{NQ}_{r,d}(t) = C_{r,n}(\mathsf{rounds}_d(t)).$$

For every original typed, d-bounded term in a well-formed context, the source of this quotation is its full normal form, reachable by original reduction and retaining its original type. The resulting closed HOAS code is normal and has type $\mathsf{HOAS}_{r,n}$. For two such sources in the same context, possibly with different sufficient budgets and types, the checked theorem gives

$$\mathsf{NQ}_{r,d}(t) =_\beta \mathsf{NQ}_{r',e}(u) \quad \Longleftrightarrow \quad r = r' \text{ and } t =_\beta u.$$

The automatic variant uses $d(t)$ and covers every closed original typed term without a supplied bound or normalization-success hypothesis. It identifies beta-convertible sources, unlike the raw quotation, which retains exact syntax. In particular, canonicalization is not being mistaken for syntax-level injectivity.

The normalization stage inherits the operation cost from the introduction. The additional checked output-size bound, for $L = \mathsf{maxRankLiteral}(t)$ and $N = \mathsf{nodes}(t)$, is

$$\mathsf{nodes}(\mathsf{NQ}_{r,d}(t)) \le 34 + 2n + (2L + 5)\,\mathrm{tower}_d(N).$$

The factor involving L accounts for unary label data in this HOAS representation. This is an output-size theorem, not a charged runtime for HOAS construction or numeric serialization. The canonicalizer is an executable Rocq function on the original syntax; it is not yet a PTS term implementing the conditional numerical program from the introduction.

7.1 Open contexts, canonical types and hereditary substitution

The canonicalizer now covers **every original well-typed open term**. Let $L(\Gamma)$ and $S(\Gamma)$ be the maximum literal label and maximum node count among the declarations of Γ, with empty maxima zero. Define executable metalevel functions

$$d(\Gamma, t) = 1 + \max\{L(\Gamma), \mathsf{maxRankLiteral}(t)\} + \max\{S(\Gamma), \mathsf{nodes}(t)\},$$
$$N_\Gamma(t) = \mathsf{rounds}_{d(\Gamma,t)}(t).$$

`OpenCanonical.budget_sound` proves this bound sufficient from original well-formedness and typing. `nf_correct` proves full normality, reduction from t, original type preservation and success of the bounded normalizer. The function consumes syntax, not an erased typing proof or a supplied normalization certificate. Its correctness requires original well-typedness; arbitrary raw input need not be normalized by these finite rounds. The normalization-stage cost is the earlier tower bound at $d(\Gamma, t)$; no new runtime bound for context scanning or code construction is claimed.

For originally typed terms, possibly in different well-formed contexts of the same scope,

$$N_\Gamma(t) = N_\Delta(u) \quad \Longleftrightarrow \quad t =_\beta u.$$

This compares original scoped syntax, not denotations of variables in unrelated environments. In particular, changing a valid typing context or sufficient normalization budget cannot change a term's normal form.

Let $\sigma : \Gamma \Rightarrow \Delta$ be an **actual original typed substitution**: $\Delta \vdash \sigma(i) : \sigma(\Gamma!!i)$ for each i. Set

$$\overline{\sigma}(i) = N_\Delta(\sigma(i)), \qquad H_{\Delta,\sigma}(t) = N_\Delta(t[\sigma]).$$

Theorem 7.1 (Full-PTS canonical substitution)**.** *For well-formed original contexts, $\Gamma \vdash t : T$ and original typed $\sigma : \Gamma \Rightarrow \Delta$, the normalized substitution $\overline{\sigma}$ is again originally typed, and*

$$N_\Delta(t[\sigma]) = N_\Delta\big(N_\Gamma(t)[\overline{\sigma}]\big),$$
$$\Delta \vdash H_{\Delta,\sigma}(t) : H_{\Delta,\sigma}(T).$$

For original typed $\tau : \Delta \Rightarrow \Theta$, composition gives

$$H_{\Theta,\tau}\big(H_{\Delta,\sigma}(t)\big) = H_{\Theta,\rho}(t), \qquad \rho(i) = H_{\Theta,\tau}(\sigma(i)).$$

Identity substitution returns $N_\Gamma(t)$.

Proof. Reduction commutes with simultaneous substitution in both the term and its entries. Normalizing an entry also reduces its dependent lookup type; original validity, preservation and conversion establish `TypedSub` for the normalized entries. The substituted terms are originally typed and convertible, so normal-form uniqueness yields literal equality. Validity supplies the exact sort needed to normalize the result type too, without cumulativity. Composition follows from ordinary substitution composition and these normalization equations. □

`CanonicalSubstitution` proves these statements for the **full original calculus**, including dependent products, sorts, conversion and annotations. The outer normalization cannot be dropped. A checked regression uses

$$\Gamma = (f : \Pi X : \mathcal{U}_0.\mathcal{U}_0), \quad A = f\,\mathcal{U}_0, \quad \sigma(f) = \lambda X : \mathcal{U}_0.X.$$

The source type and substitution entry are both normal and originally typed, but $A[\sigma] = (\lambda X : \mathcal{U}_0.X)\,\mathcal{U}_0$ is a redex with normal form $\mathcal{U}_0$. Normalizing entries alone does not make ordinary substitution canonical.

Contexts can also be normalized without changing the term being typed:

$$\mathsf{NC}(\varnothing) = \varnothing, \qquad \mathsf{NC}(\Gamma, A) = \mathsf{NC}(\Gamma), N_\Gamma(A).$$

`CanonicalContexts` constructs original well-formedness and actual identity substitutions in **both directions**. For well-formed Γ,

$$\Gamma \vdash t : T \quad \Longleftrightarrow \quad \mathsf{NC}(\Gamma) \vdash t : T.$$

The terms t, T, including annotations, are literally unchanged. The declarations become normal; context normalization is idempotent and reachable by original context reduction. Computing $N(t)$ in either context gives the same result. Formation ranks come from the original `Type`-valued well-formedness witness, not elimination of an existential `Prop` proof.

We therefore obtain concrete **distinct** syntax and type-index maps:

$$C_{r,n}(t), \qquad J_{r,\Gamma}(A) = C_{r,n}(N_\Gamma(A)).$$

`CanonicalTypeIndex` proves that the former retains literal annotated syntax, whereas equality of the latter, on originally typed inputs at fixed r, is exactly original beta conversion. Canonical indices satisfy the hereditary substitution and composition equations after quotation. On the convertible types $(\lambda X : \mathcal{U}_0.X)\,\mathcal{U}_0$ and $\mathcal{U}_0$, faithful codes differ while canonical indices agree; the faithful code of the first is not even convertible to its own canonical index. These maps genuinely avoid the shared-map obstruction.

Both maps produce closed inhabitants of the **homogeneous HOAS code type**; $J(A)$ is not itself a source type at the universe of A. Hereditary substitution here acts on original syntax before quotation, not by running an object-language program on a code. This completes a full-PTS canonical context/index algebra, but not an internal operation $\mathsf{El}(J(A)) =_\beta A$ or a translation of every dependent family to abstract represented arguments. These remain object-language obligations, not consequences to assume from normalization. No general PHOAS impossibility follows.

7.2 A full dependent weak interpretation with faithful syntax

There is nevertheless an actual closed indexed PTS decoder for a weaker representation. Normalize both the source and its type **during external quotation**, and retain the original raw HOAS tag to preserve syntax. Put $C_r = \mathsf{HOAS}_{r,0}$ and, for $A : \mathcal{U}_s$, define

$$\begin{aligned} W_r(A) &= (A \to C_r \to A) \to A, \\ \mathsf{pack}_r(A, u, c) &= \lambda h : A \to C_r \to A.\, h\,u\,c, \\ \mathsf{select}_r(A) &= \lambda x : A.\lambda c : C_r.x, \\ \mathsf{WDec}_{r,s} &= \lambda A : \mathcal{U}_s.\lambda q : W_r(A).\, q\,(\mathsf{select}_r(A)). \end{aligned}$$

Its exact original typings are

$$\begin{aligned} W_r(A) &: \mathcal{U}_{\max(s+2,r+5)}, \\ \mathsf{WDec}_{r,s} &: \Pi A : \mathcal{U}_s.\, W_r(A) \to A, \\ (\Pi A : \mathcal{U}_s.\, W_r(A) \to A) &: \mathcal{U}_{\max(s+3,r+6)}. \end{aligned}$$

For a closed original judgment $t : A$, let $\mathsf{nf}(v) = \mathsf{rounds}_{d(v)}(v)$ and define

$$Q_r(t : A) = \mathsf{pack}_r(\mathsf{nf}(A), \mathsf{nf}(t), C_{r,0}(t)).$$

Theorem 7.2 (Full closed weak interpretation)**.** *Every original closed derivation $\vdash t : A$ has a sort witness $\vdash A : \mathcal{U}_s$. The computed $Q_r(t : A)$ is fully normal, has original type $W_r(\mathsf{nf}(A))$, and satisfies*

$$\mathsf{WDec}_{r,s}\, \mathsf{nf}(A)\, Q_r(t : A) \longrightarrow_\beta^* \mathsf{nf}(t) =_\beta t.$$

The applied decoder has original type A. Moreover, conversion between two such normal packages, even with different supplied source types, implies equality of their carrier indices and literal equality of their annotated source terms.

Proof. Original type validity supplies s; normalization and preservation give $\mathsf{nf}(A) : \mathcal{U}_s$ and $\mathsf{nf}(t) : A$. Original conversion then types the payload at $\mathsf{nf}(A)$. Both payload and type are normal; the raw HOAS tag is normal independently of source reducibility. The package body is headed by its bound variable, so the whole quotation, including its annotation, is normal. The decoder performs five administrative beta steps to the payload. Its result can be converted back to type A. Conversion between normal packages implies their literal equality; extracting their tags and applying raw-HOAS fidelity recovers the exact source syntax. □

`WeakFullInterpretation.full_closed_weak_interpretation` proves coverage from original closed typing alone. `quote_conversion_injective` proves the separate fidelity claim. The supplied source type or derivation is not asserted injectively recoverable: only the annotated term is tagged. The quote is a total external syntax computation; its normality theorem has the original typing premise. The decoder's s is an **exact** sort rank, obtained propositionally by type validity, not a larger bound used via nonexistent cumulativity.

This construction includes dependent products, sort terms, reducible annotations and all other original closed typed syntax. It requires no elementary-representability assumption. But its computation occurs in the external quoter, and its decoder ignores the HOAS tag. It is therefore a normal, faithful **weak semantic-payload interpretation**, not an internal interpreter of pure HOAS and not Brown–Palsberg strong unquotation of the unreduced source. The literal obstruction is avoided because the normal payload can itself contain the source's literal labels. The computational challenge left by the introduction is to move normalization and typed reification into the object program instead of supplying this payload.

8 Independent Brown–Palsberg PHOAS investigation

The paper's Figure 3 abstracts a type interpretation $F : * \to *$. For the arrow fragment its pretranslation uses $\widehat{A \to B} = F\widehat{A} \to F\widehat{B}$, while the representation of a term of type A

has carrier $F\widehat{A}$. Its abstraction and application handlers therefore involve $F(FA \to FB)$. Unquotation specializes this same F to the type identity.

Theorem 8.1 (Obstruction to the literal single-family interface)**.** *Let $F : \Pi A : \mathcal{U}_a.\mathcal{U}_b$. If $A, B : \mathcal{U}_a$ and $F(FA \to FB)$ has any original PTS type, then $a = b + 1$. If the same family type admits the identity $\lambda A : \mathcal{U}_a.A$, then $a = b$. Thus these requirements cannot hold simultaneously, even when different input and output ranks are allowed.*

Proof. Application inversion and uniqueness of typing force the argument $FA \to FB$ to inhabit $\mathcal{U}_a$. Both FA and FB inhabit $\mathcal{U}_b$, so the original product rule places that arrow in $\mathcal{U}_{b+1}$. Uniqueness of sort ranks gives $a = b + 1$; conversion cannot change it. The literal identity already has type $\Pi A : \mathcal{U}_a.\mathcal{U}_a$. Uniqueness of typing followed by product and sort conversion inversion therefore gives $a = b$. The two equations contradict each other. □

`BrownPalsberg.no_single_family_identity_interface` proves this statement from actual original typing judgments, including arbitrary alternative types used in application derivations. `no_literal_interface` specializes the arguments to the explicitly constructed closed type $A_a : \mathcal{U}_a$, so the obstruction is not a vacuous lack of source types. This is independent of the fixed-head evaluator theorem. It excludes the literal common-family interface, not every possible PHOAS encoding.

8.1 A rank-indexed repair of the type interpretation

For the independent simply typed source, replace the one family by external parameters $F_i : \Pi A : \mathcal{U}_i.\mathcal{U}_i$ and a ground carrier $o : \mathcal{U}_0$. Define

$$\begin{aligned} \widehat{o} &= o, \\ \widehat{A \to B} &= F_a\widehat{A} \to F_b\widehat{B}, \quad a = \mathrm{ord}(A), \quad b = \mathrm{ord}(B), \\ \mathcal{I}_F(A) &= F_{\mathrm{ord}(A)}\widehat{A}. \end{aligned}$$

Both $\widehat{A}$ and $\mathcal{I}_F(A)$ inhabit precisely $\mathcal{U}_{\mathrm{ord}(A)}$. The executable `pretype` and `interpreted_type` have original typing proofs, strong recovery to the ordinary source type code under $F_i = \lambda A : \mathcal{U}_i.A$, and proved finite dependence on indices at most $\mathrm{ord}(A)$. These are syntactic functions generating PTS terms; the index i is external, not a new PTS sort variable.

With $c = \max(a + 1, b)$, the corresponding polymorphic handlers are

$$\begin{aligned} \mathsf{Abs}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : \mathcal{U}_b.\, (F_aA \to F_bB) \to F_c(F_aA \to F_bB), \\ \mathsf{App}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : \mathcal{U}_b.\, F_c(F_aA \to F_bB) \to F_aA \to F_bB. \end{aligned}$$

Both handler types inhabit $\mathcal{U}_{c+1}$; `indexed_handler_type_sorts` proves their formation. When the families are identities, the actual original lambda terms

$$\begin{aligned} \mathsf{abs}^{\mathrm{id}}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : \mathcal{U}_b.\lambda f : A \to B.f, \\ \mathsf{app}^{\mathrm{id}}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : \mathcal{U}_b.\lambda f : A \to B.\lambda x : A.f\, x \end{aligned}$$

inhabit these exact interpreted handler types by conversion.

`indexed_identity_handlers_typing` proves their original typing. The two strong beta equations are `indexed_abs_recovers` and `indexed_app_recovers`. Each is a closed original term

at its external pair (a, b). This genuinely repairs the local single-family rank conflict: the outer arrow uses F_c, not F_a or a universal F. It does not by itself supply a closed quotation package or a dependent conversion theorem.

8.2 Source-type-indexed HOAS term prequotation

The new `TypedHOAS` construction uses the independent computational source derivations described below. Unlike the raw-AST representation, its result type records the source type. Write $I(A) = \mathcal{I}_F(A)$ and suppose specialized operations are supplied with types

$$\mathsf{abs}_{A,B} : (I(A) \to I(B)) \to I(A \to B), \qquad \mathsf{app}_{A,B} : I(A \to B) \to I(A) \to I(B).$$

The executable derivation-directed prequotation is

$$\begin{aligned} q(x) &= \rho(x), \\ q(\lambda x : |A|.t) &= \mathsf{abs}_{A,B}(\lambda x : I(A).q(t)), \\ q(f\, u) &= \mathsf{app}_{A,B}\, q(f)\, q(u), \end{aligned}$$

where the source derivation supplies A, B and all binder lifts are explicit in the original scoped syntax. The theorem `prequote_typed` derives $q(t) : I(C)$ from independent source evidence $t : C$, a correctly interpreted source-variable environment, and the stated family and operation typings. The premises type the algebra, not its recursively constructed output.

The bounded variant uses the computable budget $K(D) = \max(\mathrm{ord}(C), b(D))$ for a derivation $D : t : C$, where $b(D)$ is the application budget defined below. The theorem `prequote_bounded_typed` requires family laws only at $i \le K(D)$ and case laws only when $\mathrm{ord}(A \to B) \le K(D)$. Including the result order matters: a high-order identity lambda can have application budget zero. Bounded case laws still quantify over source types, not merely a finite list of monomorphic variables. The finite implementation below specializes the rank-pair-polymorphic handlers above.

Specializing the families to the type identities and the operations to the actual closed Brown–Palsberg handlers gives strong recovery of the original annotated term (`identity_interpretation_recovers`). This is also related to actual replacement of already generated syntax: `prequote_substitution` proves that simultaneous substitution commutes with prequotation, provided it sends the embedded source variables to the stated new variables. The theorem `substitution_identity_recovers` then reduces that substituted prequote to the source under the new renaming, assuming the substitution actually supplies the specified identity families and handlers. It does not assume that the quotation itself recovers, nor assert existence of the missing finite substitution telescope. Annotation recovery is part of the original reduction proof.

`TypedHOASAlgebra` proves original typing of the actual two-application specializations of the closed identity handlers. Its theorem `typed_identity_interpretation` combines original typing at the ordinary source type with strong recovery, for every independent source derivation in an originally typed source environment. This concrete identity interpretation is not asserted normal: normal prequotation and subsequent identity interpretation are different phases.

In `TypedHOASFaithful`, neutral variable families and handlers give full annotation normality (`prequote_normal`). A structural eraser separately strips interpreted type applications and

the two constructor patterns; `erase_prequote` yields exactly rename(ρ, t). Consequently `prequote_injective` recovers literal annotated source syntax when ρ is injective, even when the two derivations, result types and handler palettes differ. `prequote_conversion_injective` strengthens this to comparison by original beta conversion of the neutral prequotes. This is not proof irrelevance for derivation evidence.

These earlier results concern parameterized prequotation. The generic typing theorem and the variable-palette normality theorem cannot be combined by silently assuming infinitely many differently typed variables in one finite context. The following construction discharges that obligation with an actual finite context and actual typed parameter substitutions. Nor is every neutral interpretation family impossible in a finite context: one $h : \mathsf{I}_0$ already gives $F_r = h\,\mathcal{U}_r$, because $\mathcal{U}_r : \mathcal{U}_0$. The construction below uses separate variable families, not this alternative.

8.3 Closing the finite rank-indexed PHOAS representation

For each external k, bind $o : \mathcal{U}_0$, the $k{+}2$ families $F_0, \ldots, F_{k+1}$, and two polymorphic handlers $\mathsf{Abs}_{a,b}, \mathsf{App}_{a,b}$ for each $0 \le a, b \le k$. The handler types are the indexed types displayed above. This deliberately includes some unused pairs. Their arrow rank can be $k+1$, which explains the extra family F_{k+1}. The finite parameter context Δ_k has

$$1 + (k+2) + 2(k+1)^2$$

binders. Its original well-formedness, all annotation normality, and the actual ground, family and handler lookup typings are proved. It depends on k, not on the number of source type shapes or the syntax of a particular program. There are infinitely many simple types at fixed order; the handlers cover them by ordinary PTS type application, not by adding one variable for each type.

Specialize a handler at the two pretypes:

$$\mathsf{abs}_{A,B} = \mathsf{Abs}_{a,b}\,\widehat{A}\,\widehat{B}, \qquad \mathsf{app}_{A,B} = \mathsf{App}_{a,b}\,\widehat{A}\,\widehat{B}, \qquad a = \mathrm{ord}(A),\ b = \mathrm{ord}(B).$$

These applied handlers remain neutral. `PHOASAlgebra` proves their actual original typing and that of the resulting derivation-directed prequotation. `PHOASFaithful` accounts for the two extra type arguments in a new eraser; it proves full normality and literal annotated-source injectivity, also when comparing quotes by beta conversion. No disjointness of handler and family variable names is assumed by the generic eraser theorem.

For a term-closed independent source derivation $D : t : A$ whose sole free parameter is o, and $K(D) \le k$, put

$$\begin{aligned} \mathsf{PCode}_{k,A} &= \Pi\Delta_k.\,\mathcal{I}_F(A), \\ \mathsf{PQuote}_k(D) &= \lambda\Delta_k.\,q_F(D). \end{aligned}$$

Here $\Pi\Delta_k$ and $\lambda\Delta_k$ are notation for finite lists of **original** products and annotated lambdas; they are not added syntax. `PHOASTelescope` implements their closure and actual sequential application. Its application-typing theorem uses a typed substitution for the supplied arguments, not subject expansion from a known reduct. `PHOASPackage` constructs the concrete context and code. The computed quote is closed, normal, and originally typed at $\mathsf{PCode}_{k,A}$.

At fixed k, conversion of quotes implies literal equality of their annotated sources, even for different source derivations and result types.

`PHOASRecovery` proves that replacing the used families by their type identities and the handlers by their actual closed identity interpretations reduces the prequote to the original source. Its hypotheses concern only the ranks and rank pairs within $K(D)$; no correctness property is required of fallback entries outside the finite palette. This is a theorem about actual replacement of generated syntax, not about separately generating a second term with different parameters.

The parameter substitution is itself realized by an actual closed decoder. Writing $\boldsymbol{\iota}_k$ for the finite list of type identities and identity Abs/App handlers in the telescope's binder order, put

$$\mathsf{PDec}_{k,A} = \lambda q : \mathsf{PCode}_{k,A}.\lambda o : \mathcal{U}_0.q\, o\, \boldsymbol{\iota}_k.$$

Theorem 8.2 (Closed finite PHOAS unquotation)**.** *For every independent term-closed Church-STLC derivation $D : t : A$ and $k \geq K(D)$, the actual original closed terms satisfy*

$$\begin{aligned} \mathsf{PQuote}_k(D) &: \mathsf{PCode}_{k,A}, \\ \mathsf{PDec}_{k,A} &: \mathsf{PCode}_{k,A} \to (\Pi o : \mathcal{U}_0.|A|), \\ \mathsf{PDec}_{k,A}\, \mathsf{PQuote}_k(D) &\longrightarrow_\beta^* \lambda o : \mathcal{U}_0.t. \end{aligned}$$

The quote is normal and injective on original annotated source syntax at fixed k, even under comparison by target beta conversion.

`PHOASDecoder.closed_phoas_interpretation` proves the typing and strong recovery claims; the separate package theorem proves fidelity. The ground abstraction is inside the decoder, not an external operation appended to evaluation. All finite parameter-typing and substitution premises are discharged by the constructed context and identity arguments. Code-type formation is proved independently of source inhabitation. The current sort theorems provide a universe witness, not a separately verified closed-form rank formula.

This decoder is indexed by **both k and the full simple type** A, not by k and $\mathrm{ord}(A)$ alone. At fixed (k, A) it is the same raw PTS program for every source derivation within budget, independent of program size; it is not chosen per source term. Internalizing the type interpretation as an additional argument, to remove the full-A external index, remains a further task.

The executable regression quotes $\lambda x : o.(\lambda y : o.y)x$ with computed budget one. Its actual closed PHOAS package computes to a normal term, while its decoder strongly recovers the original still-reducible source, including the outer ground abstraction. No normal form or semantic value is precomputed and stored in this quotation. This completes the finite PHOAS construction for the independent STLC source, not for all original dependent PTS syntax.

This is a Church-style, algebra-parametric typed HOAS construction inside PTS-rk. The term PHOAS here emphasizes that the representation abstracts its interpretation families and operations. It should not be confused with adding a native Rocq inductive datatype and calling that an internal PTS representation. In Chlipala's PHOAS[3], the variable family

[3] `https://adam.chlipala.net/papers/PhoasICFP08/`

is likewise an explicit parameter, but that work's metatheoretic treatment is not a theorem about all inhabitants of these PTS code types. Our proofs concern the actual quoted outputs; they assume no parametricity axiom or no-junk theorem for arbitrary code inhabitants.

8.4 Genuinely dependent indexed handlers

The local rank-indexed repair is not limited to constant codomains. Let $F_i : \mathcal{U}_i \to \mathcal{U}_i$, $A : \mathcal{U}_a$ and $B : (F_a A) \to \mathcal{U}_b$, and set $c = \max(a+1, b)$ and

$$D_F(A, B) = \Pi x : F_a A.\, F_b(B\, x).$$

The actual original product rule gives $D_F(A, B) : \mathcal{U}_c$. Consequently the two dependent constructor interfaces are

$$\begin{aligned} \mathsf{DAbs}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : (F_a A \to \mathcal{U}_b).\, D_F(A, B) \to F_c(D_F(A, B)), \\ \mathsf{DApp}_{a,b} &= \Pi A : \mathcal{U}_a.\Pi B : (F_a A \to \mathcal{U}_b).\, F_c(D_F(A, B)) \to D_F(A, B). \end{aligned}$$

Both inhabit exactly $\mathcal{U}_{c+1}$. In particular, the codomain of an applied constructor $h_{\mathrm{app}} : \mathsf{DApp}_{a,b}$ really depends on the represented argument:

$$h_{\mathrm{app}}\, A\, B\, q_f\, q_u : F_b(B\, q_u).$$

At identity families, the closed original handlers

$$\begin{aligned} \mathsf{dabs}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : A \to \mathcal{U}_b.\lambda f : (\Pi x : A.Bx).f, \\ \mathsf{dapp}_{a,b} &= \lambda A : \mathcal{U}_a.\lambda B : A \to \mathcal{U}_b.\lambda f : (\Pi x : A.Bx).\lambda x : A.f\, x \end{aligned}$$

inhabit these exact interpreted interfaces by proved beta conversion. Their applications reduce to f and $f\, u$, respectively. `DependentPHOAS` proves formation, operation typing, identity specialization and these reduction equations. It also derives the original type $B\, u$ of the actual applied identity App handler directly, before reducing it. No eta conversion is used.

This is positive dependent infrastructure, but the supplied family $B : (F_a A) \to \mathcal{U}_b$ has not yet been constructed recursively from every source dependent family $B_0 : A \to \mathcal{U}_b$. The domains differ while F_a is abstract. Treating this input as already translated would assume away the central coherence obligation.

For the candidate convention that interprets sort literals by themselves, a source type $T : \mathcal{U}_b$, regarded as a source term, has representation carrier $F_0(\mathcal{U}_b)$. Using that representation as an annotation requires a separate operation

$$\mathsf{El}_b : F_0(\mathcal{U}_b) \to \mathcal{U}_b.$$

Its type inhabits $\mathcal{U}_1$, and the type identity at rank b is an actual inhabitant at identity families. Both facts and its beta equation are checked. Merely binding an abstract El_b does not give it the definitional conversion laws required by a dependent quoter.

8.5 A precise coherence obstruction, not a PHOAS impossibility

Theorem 8.3 (Faithful normal syntax cannot also be a semantic type index)**.** *Let a fixed externally indexed domain contain the two closed original types*

$$T_1 = (\lambda X : \mathcal{U}_0.X)\,\mathcal{U}_0, \qquad T_0 = \mathcal{U}_0.$$

No one map Q on that domain can simultaneously have normal outputs, be injective on literal syntax, and preserve original beta conversion. The same holds if a separate conversion-preserving index map J is required to satisfy $Q(T) =_\beta J(T)$ pointwise.

Proof. Both types inhabit $\mathcal{U}_0$, and $T_1 \longrightarrow_\beta^* T_0$ while $T_1 \neq T_0$. Conversion preservation makes their normal images convertible, hence equal by original confluence. Injectivity gives the contradictory source equality. For the two-map variant, pointwise conversion transfers conversion preservation from J to Q. □

`PHOASConversion` proves the statement at any target scope and external index. Its two witnesses already fit one independently specified domain: type rank zero, literal ceiling zero, at most five nodes, and sufficient redex bound six. Increasing those indices cannot repair the three simultaneous requirements once both witnesses occur at the same index. The argument does **not** force two genuinely different maps to agree. Canonical semantic indices plus faithful syntax codes, or another explicit coherence mechanism, lie outside the obstruction.

There is also a concrete source regression: the neutral PHOAS quotes of $\lambda x : o.(\lambda y : o.y)x$ and $\lambda x : o.x$ are provably not convertible, although the sources differ by one beta step. This is expected of faithful quotation, not a defect in its interpretation theorem. For dependent syntax, a quotation map therefore cannot simply be reused unchanged as the map interpreting type conversion. This is the precise obstruction established here; arbitrary indexed dependent PHOAS has not been ruled out.

9 Executable structural quotation for the simply typed fragment

The existing independent source `ChurchSTLC` has a distinguished ground type variable $o : \mathcal{U}_0$, simple types σ, and annotated lambda terms. It has its own typing rules. Its embedding uses

$$|o| = o, \qquad |\sigma \to \tau| = |\sigma| \to |\tau|, \qquad \mathrm{ord}(\sigma \to \tau) = \max(\mathrm{ord}(\sigma) + 1, \mathrm{ord}(\tau)).$$

All type codes are beta-normal, and $|\sigma|$ inhabits $\mathcal{U}_{\mathrm{ord}(\sigma)}$ in the target context. No source rule depends on conversion of terms inside types.

Fix a budget d bounding the order of the **function type of every application** in a source typing derivation. It is not enough to bound the final result type. For an example entirely within STLC, put $\sigma_0 = o \to o$, $w_0 = \lambda z : o.z$, $\sigma_{N+1} = \sigma_N \to \sigma_0$, and $w_{N+1} = \lambda x : \sigma_N.w_0$. Then $(\lambda x : \sigma_N.w_0)\,w_N$ has the fixed result type σ_0, while its function and argument orders grow without bound. The independent relation `bounded_type` adds only this numerical bound to the original STLC rules; `bounded_type_source` forgets it, and `every_source_has_budget` proves that every source derivation has some finite budget. No quotation or recovery property defines the source domain.

Introduce variables $b_0 : \mathsf{I}_0, \ldots, b_d : \mathsf{I}_d$. The prequotation is the following derivation-directed structural translation:

$$\begin{aligned} q(x) &= x, \\ q(\lambda x : |\sigma|.t) &= \lambda x : |\sigma|.q(t), \\ q(f\ u) &= b_r\,|\sigma \to \tau|\,q(f)\,q(u), \qquad r = \mathrm{ord}(\sigma \to \tau), \end{aligned}$$

where the last case uses the source judgment $f : \sigma \to \tau$. The formal version supplies all scope changes and binder shifts explicitly. No application in $q(t)$ has a lambda at its head: each translated application is headed by a neutral blocker. The remaining annotations are normal simple-type codes. Thus the quotation is normal for the **full original PTS reduction**, including annotation reduction.

Typing follows from $b_r\,|\sigma \to \tau| : (|\sigma| \to |\tau|) \to (|\sigma| \to |\tau|)$ and the translated subderivations. Substituting id_r for each blocker gives strong recovery to the original annotated source term by induction. Syntactic injectivity is proved at each fixed budget with a structural eraser for the administrative blocker pattern. Recovery up to beta conversion alone would not establish injectivity, since distinct source terms may be beta-convertible.

The Rocq module `RankedQuotation` implements these claims for arbitrary bounded derivations and explicitly typed blocker contexts. Its original existence theorem is relational, in Rocq `Prop`. The theorem `finite_budget_quotation` supplies the actual finite blocker context, and `finite_package_injective` proves injectivity of the resulting packages. `every_source_has_normal_representation` covers every independent STLC derivation at some budget.

The new `ExecutableQuotation` module supplies an executable external algorithm. Its source derivation is an inductive datatype in Rocq `Type`, with the independent variable, lambda, and application rules, indexed by the original annotated source term and its simple type. No constructor assumes target PTS typing, quotation, or recovery. The theorem `source_derivation` relates its inhabitation to the earlier independent `ChurchSTLC.has_type` judgment. This is not an illicit elimination from a proof in `Prop` into computational data: the executable function consumes the derivation data explicitly. The coverage theorem remains a proposition; it is not a source type-inference algorithm.

Structural recursion computes both prequotation and a sufficient budget:

$$\begin{aligned} b(D_{\mathrm{var}}) &= 0, & b(D_{\mathrm{lam}}) &= b(D_{\mathrm{body}}), \\ b(D_{\mathrm{app}}) &= \max\{\mathrm{ord}(\sigma \to \tau), b(D_f), b(D_u)\}. \end{aligned}$$

The executable prequotation follows exactly the displayed structural clauses. Its relationship to the existing relation is two-way: a related prequote exists exactly when some computational source derivation produces that prequote with budget at most d. Conversely, each supplied derivation produces a related prequote at every sufficient budget. Thus executable normality, original PTS typing, and strong recovery reuse the proved relational construction, without assuming its output as an input witness. After packing the computed blocker telescope, equality of executable packages at any fixed sufficient budget implies exact equality of the original annotated source terms. This injectivity statement does not identify beta-convertible but syntactically different sources. The implementation includes a reducible source $\lambda x : o.(\lambda y : o.y)\,x$: its budget computes to 1, its closed quote computes to a normal

term, and the closed decoder recovers the original, still reducible source. A separate computation checks subsequent normalization. These examples test the executable path; the coverage and correctness theorems quantify over all source derivations.

The finite-telescope interface abstracts those blockers in descending order:

$$\begin{aligned}\mathsf{Code}_{\le d}(A) &= \Pi b_d : \mathsf{I}_d. \cdots \Pi b_0 : \mathsf{I}_0.A,\\ \ulcorner t \urcorner_d &= \lambda b_d : \mathsf{I}_d. \cdots \lambda b_0 : \mathsf{I}_0.q(t),\\ \mathsf{D}^{\mathrm{tel}}_{d,s} &= \lambda A : \mathcal{U}_s.\lambda q : \mathsf{Code}_{\le d}(A).q\,\mathsf{id}_d \cdots \mathsf{id}_0.\end{aligned}$$

For $A : \mathcal{U}_s$, direct rank calculation gives $\mathsf{Code}_{\le d}(A) : \mathcal{U}_{\max(d+2,s)}$ and a decoder type of rank $\max(d+3, s+1)$. `TelescopeDecoder.family_decoder_typing` checks the actual decoder, with its exact rank expressed by the recursive function `package_rank`. The theorem `fixed_decoder_bounded_quotation` uses the same decoder for every source with budget d and result order s.

Closing over the distinguished ground type also has an internal solution. Let

$$\begin{aligned}K_s &= \Pi o : \mathcal{U}_0.\mathcal{U}_s,\\ C_d(P) &= \Pi o : \mathcal{U}_0.\mathsf{Code}_{\le d}(P\,o),\\ \mathsf{D}^{\mathrm{cl}}_{d,s} &= \lambda P : K_s.\lambda q : C_d(P).\lambda o : \mathcal{U}_0.(q\,o)\,\mathsf{id}_d \cdots \mathsf{id}_0.\end{aligned}$$

This is one closed PTS term for each fixed pair (d, s), with type $\Pi P : K_s.C_d(P) \to (\Pi o : \mathcal{U}_0.P\,o)$. For a term-closed source $t : \sigma$ with its single free type variable o, put $P_\sigma = \lambda o : \mathcal{U}_0.|\sigma|$ and $Q = \lambda o : \mathcal{U}_0.\ulcorner t \urcorner_d$. The closed representation Q is normal, and the actual decoder application satisfies

$$\mathsf{D}^{\mathrm{cl}}_{d,s}\,P_\sigma\,Q \longrightarrow^*_\beta \lambda o : \mathcal{U}_0.t, \qquad s = \mathrm{ord}(\sigma).$$

`ClosedDecoder.closed_fixed_decoder_bounded_quotation` proves original typing and strong recovery for this closed interface. The ground-type abstraction is inside the decoder; it is not an external evaluation step. Every independent STLC source derivation has a finite budget, so this is an infinite fragment result. The fragment does not include all PTS-rk syntax or the decoder's own polymorphic code, and no self-applicability claim follows.

10 Why dependent quotation is still open

The rank-indexed identities solve a rank-selection problem. They do not by themselves solve dependent conversion. Consider the well-formed context

$$A : \mathcal{U}_0, \quad B : \Pi x : A.\mathcal{U}_0, \quad a : A, \quad f : \Pi x : A.B\,x.$$

The source term $f\,((\lambda x : A.x)\,a)$ can be given type $B\,a$. If its inner redex is replaced by $b_1\,(A \to A)\,(\lambda x : A.x)\,a$, that argument is neutral, so its occurrence in the result type is no longer convertible to a. Blocking other applications as well does not restore the lost conversion. This example diagnoses failure of the naive, type-preserving shallow translation; it is not a theorem that every dependent encoding is impossible.

The paper's Figure 2 term-reduction rules do not reduce inside type annotations; PTS-rk's original reduction does. Its normality theorem therefore cannot be imported as the stronger annotation-normality result required here. One cannot avoid the dependent conversion issue by leaving all type expressions untouched: original PTS-rk reduction traverses lambda annotations and products, so a reducible annotation would prevent full normality of the quote. Nor may the typing theorem be replaced by a native Rocq checker judgment, or supplied as a premise of the desired quotation theorem.

A deeper indexed construction would need jointly compatible translations of contexts, types, terms, substitution, and conversion. An explicit interpretation relation or transported representation of equality might help, but the required witnesses must be built inside the original PTS. The canonical-context, hereditary-substitution and distinct-index results above supply this compatibility for metalevel original syntax. Lifting it to an object-language dependent representation is a separate step. The extensional instantiation functions of Brown–Palsberg suggest a direction; their correctness for F_ω does not supply this dependent coherence theorem.

11 Formalization status and next obligations

The separate library is `PTSrk.SelfInterpreter`, physically in `theories/self_interpreter`. It imports the original kernel and existing normalization infrastructure; neither arithmetic interpretation nor either set-model library is a dependency. The reference ZFC development is unchanged.

The `ChurchSyntax` modules implement raw HOAS, fidelity and folds. `BrownPalsberg` proves the single-family obstruction and indexed type interpretation. The three `TypedHOAS` modules provide source-indexed prequotation, syntax fidelity and the actual typed identity interpretation. The new `PHOASAlgebra`, `PHOASFaithful`, `PHOASTelescope`, `PHOASRecovery`, `PHOASPackage` and `PHOASDecoder` modules close that construction over a finite rank-indexed palette and supply an actual strongly recovering closed decoder for each budget and simple source type. `DependentPHOAS` constructs genuinely dependent local handlers and a candidate universe decoder; `PHOASConversion` proves the limited shared-map coherence obstruction. `OpenCanonical`, `CanonicalSubstitution`, `CanonicalContexts` and `CanonicalTypeIndex` provide the full-PTS open-context canonical index algebra, with hereditary substitution equations and bidirectional original typing transport, separately from an internal evaluator. The earlier `RankedQuotation` and `ExecutableQuotation` modules retain the independent shallow STLC quotation, with actual closed decoders in `TelescopeDecoder` and `ClosedDecoder`. `RankedNormalization` connects full PTS normalization to canonical HOAS codes; `LiteralObstruction` distinguishes redex/type-rank indices from the needed literal ceiling. `WeakFullInterpretation` supplies the full closed weak semantic-payload construction, including normality and exact source fidelity. None of these modules assumes the manuscript's unformalized elementary compiler. Public endpoints and their precise boundaries are recorded in the development README and checked by `Audit`.

The development uses the pinned Rocq 9.2.0, Stdlib 9.1.0, Equations 1.3.2+9.2, OCaml 4.14.2, and Dune 3.23.1 environment. Fresh assumption reports and recursive kernel checks are compared with a pristine upstream build. No new axioms, admitted proofs, unsafe casts, modified PTS rules, or disabled checks are part of this development. Verification commands and failures are recorded separately from theorem claims.

The remaining research programme is:

1. Formalize the non-uniform elementary compiler, including the corrected modulus construction, and instantiate it with fixed-budget normalization and elementary serialization. Prove original numeral-program typings and beta equations; a native arithmetic computation is not that proof.
2. Construct actual typed HOAS-to-numeric adapters and normal-form reification with sufficient rank and literal indices. The latter must return an inhabitant of the represented dependent type, not just another code. This would strengthen the full-PTS weak construction by eliminating its externally supplied normal semantic payload.
3. Internalize the finite PHOAS decoder's full simple-type interpretation as an explicit object argument, instead of its current external A index. Preserve normality, literal fidelity and direct original typing.
4. Lift the checked full-PTS canonical context/type-index algebra into object-language dependent quotation. Construct represented dependent families and typed reification with their internal coherence laws; metalevel normal-form equality does not supply these programs.
5. Prove all five required contracts for the intended dependent source domain: original typing, full normality, injectivity under a stated annotation equality, strong recovery, and coverage of every intended source derivation at some finite external budget.

The checked full-PTS interpretation result is the weak semantic-payload construction; the strong shallow and now finite closed PHOAS unquotation results remain for the independent STLC fragment. No full dependent pure-HOAS/PHOAS evaluator or all-rank single program is claimed. The complexity argument gives a positive conditional route to numeric self-reduction, not a proof that the remaining typed reifier exists.

To rebuild this note, run `bash tools/build-self-interpreter-note.sh`. To compile the development in the pinned environment, run `dune build @theories/self_interpreter/audit`; use a fresh absolute build directory for fresh assumption output.